

[CS230] Shallow Neural Networks: Forward & Backward Propagation

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1. Deep Learning Big Picture - *Completed*

Chapter 2. Logistic Regression as a Neural Network - *Completed*

Chapter 3. Shallow Neural Networks (Current Unit)

- 3.1 Concept of Hidden Layer & Architecture - *Completed*
- 3.2 Activation Functions - *Completed*
- **3.3 Forward & Backward Propagation (Math Heavy!)**
 - * Forward: The Flow of Prediction
 - * Backward: The "Blame Game" (Gradient Calculation)
 - * The 6 Magic Equations
 - * Implementation with Dimensions Check
- 3.4 Random Initialization

Chapter 4. Deep Neural Networks - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 우리는 신경망의 '구조(Layer)'와 '스위치(Activation Function)'를 장착했습니다. 이제 자동차는 완성되었습니다. 하지만 자동차를 앞으로 달리게만 해서는 운전을 배울 수 없습니다. 사고를 났을 때(오차가 발생했을 때), 무엇이 잘못되었는지 파악하고 핸들을 돌리는 법(수정하는 법)을 배워야 합니다. 오늘 배울 **역전파(Backpropagation)**가 바로 그 과정입니다. 수학 기호가 쏟아지겠지만, 포기하지 마십시오. 이것이 딥러닝의 심장입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 딥러닝 학습 메커니즘인 '순전파'와 '역전파'를 수식과 코드로 구현합니다.

- **순전파 (Forward):** 입력 X 가 은닉층을 거쳐 출력 $\hat{y}$ 가 되는 과정을 행렬로 정의합니다.
- **역전파 (Backward):** 예측이 틀렸을 때, 비용 함수(Cost)의 기울기(Gradient)를 뒤쪽에서 앞쪽으로 계산합니다.
- **도구:** 미적분의 연쇄 법칙(Chain Rule)과 행렬의 전치(Transpose)가 왜 필요한지 이해합니다.
- **구현:** 차원(Dimension) 오류 없이 역전파 알고리즘을 코딩합니다.

2 Essential Terminology

기호	의미	비유 (역할)
$Z^{[l]}$	선형 출력 ($WX + b$)	뉴런이 받아들인 원시 점수
$A^{[l]}$	활성화 출력 ($g(Z)$)	점수를 확률/신호로 변환한 최종 리포트
$dZ^{[l]}$	오차항 ($\partial J / \partial Z$)	"얼마나 틀렸니?" (책임의 크기)
$dW^{[l]}$	가중치 기울기	"가중치를 얼마나 수정할까?"
*	요소별 곱 (Element-wise)	행렬 곱이 아니라, 같은 위치끼리 곱함

3 Core Concepts: 흐름의 이해

3.1 1. Forward Propagation (예측의 흐름)

데이터가 강물처럼 입력층에서 출력층으로 흐릅니다.

$$Input(X) \xrightarrow{W^{[1],b^{[1]}}} Hidden(A^{[1]}) \xrightarrow{W^{[2],b^{[2]}}} Output(A^{[2]})$$

이 과정은 직관적입니다. "입력받아서, 계산하고, 넘겨준다." 끝입니다.

3.2 2. Backward Propagation (학습의 흐름)

예측값($A^{[2]}$)과 실제값(Y)의 차이, 즉 **비용(Cost)**을 줄이기 위해 미분을 사용합니다. 문제는 우리가 수정하고 싶은 파라미터($W^{[1]}$)가 출력층에서 멀리 떨어져 있다는 것입니다.

□ 프로젝트 실패의 책임 소재 따지기 (The Blame Game) (직관적 비유)

여러분이 팀장(출력층)이고 프로젝트가 실패(Error)했다고 가정해봅시다.

1. **Step 1 (Output Layer):** 먼저 최종 결과물($A^{[2]}$)을 보고 "얼마나 부족했는지($dZ^{[2]}$)" 파악합니다.
2. **Step 2 (Hidden Layer):** 팀장은 자신의 실패 원인을 분석하여, 중간 관리자(은닉층, $A^{[1]}$)에게 책임을 묻습니다. "내가 준 보고서가 잘못돼서 결과가 이렇게 됐잖아!" ($dZ^{[1]}$ 전파)
3. **Step 3 (Parameters):** 중간 관리자는 다시 자신의 업무 도구($W^{[1]}$)를 탓하며 수정합니다. "이 가중치가 문제였군, 고치자." ($dW^{[1]}$ 계산)

역전파는 이처럼 오차(책임)를 뒤에서 앞으로 전달하며 파라미터를 수정하는 과정입니다.

4 Deep Dive: The 6 Magic Equations

이 6개의 수식은 딥러닝 엔지니어의 구구단입니다. 연쇄 법칙(Chain Rule)에 의해 유도됩니다.

4.1 Phase 1: 출력층 (Layer 2) - 역전파의 시작

1. 오차 계산 ($dZ^{[2]}$) 가장 직관적인 수식입니다. 예측값과 정답의 차이입니다.

$$dZ^{[2]} = A^{[2]} - Y$$

(참고: Cross-Entropy와 Sigmoid 미분이 만나면 이렇게 깔끔하게 정리됩니다.)

2. 가중치 기울기 ($dW^{[2]}$) 오차($dZ^{[2]}$)에 입력값($A^{[1]}$)을 곱합니다.

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

왜 전치(T)를 하나요? (오해 방지 가이드)

행렬 곱셈의 차원을 맞추기 위해서입니다. $dZ^{[2]}$ 는 $(1, m)$, $A^{[1]}$ 은 $(n^{[1]}, m)$ 입니다. 곱하려면 $A^{[1]}$ 을 뒤집어야 $(1, m) \times (m, n^{[1]}) = (1, n^{[1]})$ 이 되어 $W^{[2]}$ 와 크기가 같아집니다.

3. 편향 기울기 ($db^{[2]}$) 오차들의 평균입니다.

$$db^{[2]} = \frac{1}{m} \sum_{rows} dZ^{[2]}$$

4.2 Phase 2: 은닉층 (Layer 1) - 핵심 구간

4. 은닉층 오차 ($dZ^{[1]}$) 여기가 가장 어렵습니다. 출력층의 오차를 가중치 비율만큼 가져오고 (Linear), 활성화 함수의 미분값(Non-linear)을 곱합니다.

$$dZ^{[1]} = \underbrace{W^{[2]T} dZ^{[2]}}_{\text{오차 전파}} \underbrace{*}_{\text{요소별 곱}} \underbrace{g'^{[1]}(Z^{[1]})}_{\text{활성화 미분}}$$

- $W^{[2]T}dZ^{[2]}$: 출력층의 오차를 은닉층으로 역송신합니다.
- *: 행렬 곱이 아닙니다! **Element-wise product**입니다.
- $g'(Z^{[1]})$: 만약 Tanh를 썼다면 $(1 - A^{[1]2})$ 입니다.

5, 6. 파라미터 기울기 ($dW^{[1]}, db^{[1]}$) Layer 2와 동일한 패턴입니다.

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} \sum dZ^{[1]}$$

5 Implementation (Python with NumPy)

수식을 코드로 옮길 때 가장 중요한 것은 차원(Shape) 확인입니다.

```

1 import numpy as np
2
3 def backward_propagation(parameters, cache, X, Y):
4     """
5     parameters: W1, b1, W2, b2
6     cache: Z1, A1, Z2, A2 (Forward 단계에서 저장해둔 값)
7     X, Y: 입력 데이터 및 정답
8     """
9     m = X.shape[1] # 데이터 개수
10
11     # 1. 파라미터 및 캐시 로드
12     W2 = parameters["W2"]
13     A1 = cache["A1"]
14     A2 = cache["A2"]
15
16     # --- Layer 2 (Output) ---
17     # 수식 1: dZ2 = A2 - Y
18     dZ2 = A2 - Y
19
20     # 수식 2: dW2 행렬( 곱 주의: dZ2 @ A1.T)
21     dW2 = (1 / m) * np.dot(dZ2, A1.T)
22
23     # 수식 3: db2 행( 방향 합계, keepdims 필수)
24     db2 = (1 / m) * np.sum(dZ2, axis=1, keepdims=True)
25
26     # --- Layer 1 (Hidden) ---
27     # 수식 4: dZ1 계산 가장( 중요!)
28     # g'(z) for Tanh = 1 - a^2
29     # '*' 연산자는 요소별 곱(Element-wise)임에 유의
30     dZ1 = np.dot(W2.T, dZ2) * (1 - np.power(A1, 2))
31
32     # 수식 5: dW1
33     dW1 = (1 / m) * np.dot(dZ1, X.T)
34
35     # 수식 6: db1
36     db1 = (1 / m) * np.sum(dZ1, axis=1, keepdims=True)
37
38     grads = {"dW1": dW1, "db1": db1, "dW2": dW2, "db2": db2}
39     return grads

```

Listing 1: Full Backpropagation Implementation

6 Numerical Example: 계산 흐름 추적

□ 간단한 오차 역전파 예시 (실전 시나리오 & 계산)

상황: 정답 $y = 1$ 인데, 모델이 예측 $a^{[2]} = 0.8$ 을 내놓았습니다.

1. 오차 발생 ($dZ^{[2]}$): $0.8 - 1.0 = -0.2$. (0.2만큼 부족함)
2. 은닉층 전달: $W^{[2]}$ 가 0.5라고 가정합시다. 은닉층으로 오차를 보냅니다.

$$\text{전달된 오차} \approx 0.5 \times (-0.2) = -0.1$$

3. 활성화 미분 반영: 만약 은닉층 활성화 미분값이 0.5라면?

$$dZ^{[1]} = -0.1 \times 0.5 = -0.05$$

4. 결론: 은닉층의 오차는 -0.05입니다. 이 값을 줄이는 방향으로 $W^{[1]}$ 을 업데이트합니다.

7 FAQ & Pitfalls

Q1. $dZ^{[1]}$ 구할 때 왜 행렬 곱(dot)이 아니라 요소별 곱(*) 인가요?

A. 연쇄 법칙 $\frac{\partial A}{\partial Z}$ 부분 때문입니다. 활성화 함수의 미분은 각 뉴런마다 개별적으로 적용됩니다. 행렬 전체를 섞는(Linear mixing) 과정이 아니므로 같은 위치의 원소끼리만 곱해야 합니다.

Q2. 전치(T)는 언제 하나요? 외워야 하나요?

A. 외우지 마세요. 차원(Dimensions)을 그려보면 됩니다. dW 는 W 와 모양이 같아야 합니다. (n, m) 과 $(1, m)$ 을 곱해서 $(n, 1)$ 을 만들려면 뒤의 것을 뒤집어야 한다는 것이 자연스럽게 보입니다.

다음 단계 (Next Step)

고생하셨습니다. 여러분은 방금 딥러닝에서 가장 험난한 고개인 '역전파'를 넘었습니다. 이제 모델을 학습시킬 준비가 거의 다 되었습니다.

그런데, 학습을 시작할 때 가중치(W)를 처음에 어떻게 설정하느냐가 학습의 성패를 좌우한다는 사실을 아십니까? 다음 시간에는 [Practice] 세션으로, 랜덤 초기화(Random Initialization)의 중요성을 다루고, 왜 0으로 초기화하면 이 모든 역전파 알고리즘이 무용지물이 되는지 증명하겠습니다.

□ 단원 요약 (Cheat Sheet)

1. 역전파: 출력층의 오차(dZ)를 구해 입력층 방향으로 전파하며 기울기(dW)를 구한다.
2. Chain Rule: 층을 건너갈 때마다 미분값을 곱한다 (미분의 연쇄).
3. Transpose: 행렬 곱셈 시 차원을 맞추기 위해 전치 행렬(A^T)을 사용한다.
4. Element-wise: 활성화 함수의 미분값은 반드시 요소별 곱(*)으로 계산한다.